Algorithms I

BA4 • *2025–2026*

Contents

These notes cover the core algorithmique techniques of the BA4 Algorithms I course: asymptotic analysis, sorting, divide-and-conquer, heaps, binary search trees, and graph algorithms (max flow, matchings).

CHAPTER 1

Algorithms Fundamentals

1.1 Definition of an Algorithm

Definition

An **algorithm** is a well-defined computational procedure that takes some value (or set of values) as *input* and produces some value (or set of values) as *output* in a finite number of steps.

We analyse algorithms along two axes:

- **Correctness** — does it always produce the right answer?
- **Efficiency** — how many resources (time, space) does it consume?

1.2 Asymptotic Complexity

Asymptotic notations

Let $f, g : \mathbb{N} \to \mathbb{R}^+$.

- $f \in O(g)$ **(upper bound)**: $\exists\, c, n_0 > 0$ s.t. $f(n) \leq c\, g(n)$ for all $n \geq n_0$
- $f \in \Omega(g)$ **(lower bound)**: $\exists\, c, n_0 > 0$ s.t. $f(n) \geq c\, g(n)$ for all $n \geq n_0$
- $f \in \Theta(g)$ **(tight bound)**: $f \in O(g)$ and $f \in \Omega(g)$
- $f \in o(g)$ **(strict upper)**: $\lim_{n\to\infty} f(n)/g(n) = 0 oralsof(n) < g(n)$
- $f \in \omega(g)$ **(strict lower)**: $\lim_{n\to\infty} f(n)/g(n) = \infty oralsof(n) > g(n)$

Common growth rates (slowest to fastest)

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

Remark

For the **Master Theorem**: given $T(n) = a\,T(n/b) + f(n)$ with $a \geq 1$, $b > 1$:

- if $f(n) = O(n^{\log_b a - \varepsilon})$ then $T(n) = \Theta(n^{\log_b a})$
- if $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \log n)$
- if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ then $T(n) = \Theta(f(n))$

CHAPTER 2

Sorting Algorithms

2.1 Insertion Sort

Complexity

Time: $\Theta(n^2)$ worst/average, $\Theta(n)$ best (already sorted). Space: $O(1)$ in-place.

```python
def insertion_sort(A):
    for j in range(1, len(A)):
        key = A[j]
        i = j - 1
        while i >= 0 and A[i] > key:
            A[i + 1] = A[i]
            i -= 1
        A[i + 1] = key
```

```
Insertion-Sort(A, n)
    for j = 2 to n
        key = A[j]
        // Insert A[j] into the sorted sequence A[1...j - 1].
        i = j - 1
        while i > 0 and A[i] > key
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
```

Loop invariant: at the start of each iteration, `A[0..j-1]` is sorted.
Concept and objective: Sort an array

2.2 Divide and Conquer principles

Paradigm

Divide the problem into subproblems, **conquer** each recursively, then **combine** the results.

2.3 Merge Sort

Complexity

Time: $\Theta(n \log n)$ in all cases. Space: $O(n)$ auxiliary.

```python
def merge_sort(A, p, r):
    if p < r:
        q = (p + r) // 2
        merge_sort(A, p, q)
        merge_sort(A, q + 1, r)
        merge(A, p, q, r)   # merge two sorted halves
```

```
MERGE-SORT(A, p, r)
    if p < r                // check for base case
        q = rounddown((p + r)/2)     // divide
        MERGE-SORT(A, p, q) // conquer
        MERGE-SORT(A, q + 1, r) // conquer
        MERGE(A, p, q, r)   // combine

MERGE(A,p,q,r)
        n1 = q-p+1
        n2 = r-q
        let L[1...n1 + 1] and R[1 ... n2+1] be new arrays
        for i = 1 to n1
                L[i] = A[p+i-1]
        for j = 1 to n2
                R[j] = A[q+j]
        L[,1+1] = 8
        R[n2+1] = 8
        i = 1
        j = 1
        for k = p to r
                if L[i] <= R[j]
                        A[k] = L[i]
                        i = i + 1
                else A[k] = R[j]
                        j = j + 1
```

Recurrence: $T(n) = 2\,T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$ by Master Theorem (case 2). //
Concept and objective: Sort an array

2.4 Master theorem

Theorem (Master Theorem)

Let $a \geq 1$ and $b > 1$ be constants, let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n).$$

Then, $T(n)$ has the following asymptotic bounds:

- If $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some constant $\epsilon > 0$, then

$$T(n) = \Theta\left(n^{\log_b a}\right).$$

- If $f(n) = \Theta\left(n^{\log_b a}\right)$, then

$$T(n) = \Theta\left(n^{\log_b a} \log n\right).$$

- If $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for some constant $\epsilon > 0$, and if

$$a \cdot f(n/b) \leq c \cdot f(n)$$

for some constant $c < 1$ and all sufficiently large n, then

$$T(n) = \Theta(f(n)).$$

CHAPTER 3

Maximum Subarray, Matrix Multiplication and Strassen ect

3.1 Maximum Subarray optimized

Find indices $i \leq j$ maximising $\sum_{k=i}^{j} A[k]$.

Complexity

$T(n) = 2\,T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$. (Kadane's linear algorithm also exists: $\Theta(n)$.)

```
FIND-MAXIMUM-SUBARRAY(A, low, high):
    if high == low: return (low, high, A[low])
    mid = (low + high) / 2
    left  = FIND-MAXIMUM-SUBARRAY(A, low, mid)
    right = FIND-MAXIMUM-SUBARRAY(A, mid+1, high)
    cross = FIND-MAX-CROSSING(A, low, mid, high)

    if left-sum >= right-sum and left-sum >= cross-sum
    return (left-low, left-high, left-sum)
    elseif right-sum >= left-sum and right-sum >= cross-sum
        return (right-low, right-high, right-sum)
    else
        return (cross-low, cross-high, cross-sum)
```

```
FIND-MAX-CROSSING(A, low, mid, high)

  // Find a maximum subarray of the form A[i .. mid].
  left-sum = -8
  sum = 0
  for i = mid downto low
      sum = sum + A[i]
      if sum > left-sum
          left-sum = sum
          max-left = i

  // Find a maximum subarray of the form A[mid + 1 .. j].
  right-sum = -8
  sum = 0
  for j = mid + 1 to high
      sum = sum + A[j]
      if sum > right-sum
          right-sum = sum
          max-right = j

  // Return the indices and the sum of the two subarrays.
  return (max-left, max-right, left-sum + right-sum)
```

// **Concept et objectif :**

L'algorithme `FIND-MAXIMUM-SUBARRAY` utilise la méthode du *divide and conquer* (diviser pour régner). Son objectif est de trouver la sous-partie continue d'un tableau dont la somme des éléments est maximale, le sous-enesemble de nombres qui font que leur somme est maximale.

Le principe est simple :

- le tableau est séparé en deux parties,
- l'algorithme cherche récursivement la meilleure sous-partie à gauche,
- puis la meilleure à droite,
- et enfin la meilleure sous-partie qui traverse le milieu.

Parmi ces trois possibilités, il retourne celle ayant la plus grande somme.

3.2 Maximum Subarray BRUTE FORCE

```
MAXIMUM-SUBARRAY-SLOW(A[1..n]):

    B.val <- -8
    B.i <- 1
    B.j <- n

    for i <- 1 to n
        tmp <- 0

        for j <- i to n
            tmp <- tmp + A[j]

            if tmp > B.val
                B.val <- tmp
                B.i <- i
                B.j <- j

    return (B.i, B.j, B.val)
```

Running time:

$$\Theta(n^2)$$

Space usage:

$$\Theta(n)$$

($\Theta(n)$ for input + $\Theta(1)$ extra)

3.3 Matrix Multiplication naive

Standard $O(n^3)$ triple loop, or the recursive divide-and-conquer that splits each $n \times n$ matrix into four $n/2 \times n/2$ blocks: $T(n) = 8\,T(n/2) + \Theta(n^2)$, giving the same $\Theta(n^3)$ by Master Theorem (case 1).

```
SQUARE-MAT-MULT(A, B, n)
    let C be a new n x n matrix

    for i = 1 to n
        for j = 1 to n
            c_{ij} = 0
            for k = 1 to n
                c_{ij} = c_{ij} + a_{ik} * b_{kj}

    return C
```

Goal: Perform Matrix multiplication
Time complexity: $O(n^3)$
Space: $O(n^2)$

3.4 REC-MAT-MULT - other naive matrix multiplication - divide and conquer logic

```
REC-MAT-MULT(A, B, n)
    let C be a new n x n matrix

    if n == 1
        c11 = a11 * b11

    else
        partition A, B, and C into n/2 x n/2 submatrices

        C11 = REC-MAT-MULT(A11, B11, n/2) + REC-MAT-MULT(A12, B21, n/2)
        C12 = REC-MAT-MULT(A11, B12, n/2) + REC-MAT-MULT(A12, B22, n/2)
        C21 = REC-MAT-MULT(A21, B11, n/2) + REC-MAT-MULT(A22, B21, n/2)
        C22 = REC-MAT-MULT(A21, B12, n/2) + REC-MAT-MULT(A22, B22, n/2)

    return C
```

Goal: Perform matrix multiplication but here using divide and conquer logic
Time complexity: $O(n^3)$
Space: $O(n^2)$

3.5 Strassen's algorithm - better matrix multiplication

Complexity

$T(n) = 7\,T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

Key idea: compute only 7 products of submatrices (instead of 8) using the following auxiliary matrices $M_1, \ldots, M_7$, each a linear combination of blocks of A and B:

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22}), \quad M_2 = (A_{21} + A_{22})B_{11}, \quad \ldots$$

Then reconstruct $C_{11}, C_{12}, C_{21}, C_{22}$ from M_1–M_7.

```
STRASSEN-MAT-MULT(A, B, n)
    let C be a new n * n matrix
    if n == 1
        C11 = A11 * B11
    else
        partition A, B, and C into n/2 x n/2 submatrices

        M1 = (A11 + A22) * (B11 + B22)
        M2 = (A21 + A22) * B11
        M3 = A11 * (B12 - B22)
        M4 = A22 * (B21 - B11)
        M5 = (A11 + A12) * B22
        M6 = (A21 - A11) * (B11 + B12)
        M7 = (A12 - A22) * (B21 + B22)

        C11 = M1 + M4 - M5 + M7
        C12 = M3 + M5
        C21 = M2 + M4
        C22 = M1 - M2 + M3 + M6

    return C
```

Goal: Perform matrix multiplication more efficiently than the classical method
Time complexity: $O(n^{\log_2 7})$
Space: $O(n^2)$
Idea: Reduce the number of recursive multiplications from 8 to 7 by using clever linear combinations of submatrices.

3.5.1 Karatsuba Multiplication

Multiply two n-digit integers (en base b n'importe laquelle) faster than the naive $O(n^2)$ schoolbook method. Pourquoi est-ce qu'on en parle maintenant ? Parce que tout simplement ça va utiliser un peu la même logique que Strassen's algorithm parce que on va séparer en différents calculs qui vont nous permettre de résuire au final le nombre de multiplicationsà réaliser.

Split: $x = x_H b^{n/2} + x_L$, $y = y_H b^{n/2} + y_L$.

Compute only 3 multiplications:

$$p_1 = x_H \cdot y_H, \quad p_2 = x_L \cdot y_L, \quad p_3 = (x_H + x_L)(y_H + y_L)$$

then $x \cdot y = p_1 b^n + (p_3 - p_1 - p_2)\, b^{n/2} + p_2$.

Complexity

$T(n) = 3\,T(n/2) + \Theta(n) \Rightarrow \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Ceci n'estr pas un pseudocode pseudocodesque mais ça explique bien le principe de l'algo:

```
KARATSUBA-MULT(x, y, n)
    if n == 1
        return x * y

    else
        split x into xH, xL
        split y into yH, yL

        p1 = KARATSUBA-MULT(xH, yH, n/2)
        p2 = KARATSUBA-MULT(xL, yL, n/2)
        p3 = KARATSUBA-MULT(xH + xL, yH + yL, n/2)

        return p1 * b^n + (p3 - p1 - p2) * b^(n/2) + p2
```

Goal: Perform multiplication faster than the naive method
Time complexity: $O(n^{\log_2 3})$

Space: $O(1)$

CHAPTER 4

Heap data structure

Heap

Un **heap** (ou tas) est une structure de données sous forme d'arbre binaire presque complet, utilisée pour gérer efficacement des priorités.
COMMENT ON LE STOCKE DANS UN COMPUTER ??????????
Il est généralement stocké sous forme de tableau :

$$\begin{aligned} \text{LEFT}(i) &= 2i \\ \text{RIGHT}(i) &= 2i + 1 \\ \text{PARENT}(i) &= \left\lfloor \frac{i}{2} \right\rfloor \end{aligned}$$

Cette structure est très utilisée pour les files de priorité et l'algorithme de tri *heapsort*.

Max-heap property

A **max-heap** is a nearly complete binary tree where every node satisfies $A[\text{parent}(i)] \geq A[i]$.

Min-heap property

A **min-heap** is a nearly complete binary tree where every node satisfies $A[\text{parent}(i)] \leq A[i]$.

Root

The **root** is the top node of the heap. In array representation, it is at index 1. In a max-heap, it contains the largest element.

Leaf

A **leaf** is a node with no children. Leaves are located at the bottom level of the heap.

Height

The **height** of a heap is the number of edges on the longest path from the root to a leaf. A heap of size n has height $\Theta(\log n)$.

Height of a node

Height of node is the number of edges on a longest simple path from the node down to a leaf. (logiquement ça ne peut pas être plus grand que la height du tree en entier)

Shape property

A heap is a **complete binary tree**: all levels are full except possibly the last, which is filled from left to right.

4.1 MAX-HEAPIFY

Ce qu'on veut maintenant c'est un algorithm capable de prendre un heap et d'en faire un max heap donc on utilise MAX HEAPIFY **Algorithme :**

- Comparer $A[i]$, $A[\text{LEFT}(i)]$ et $A[\text{RIGHT}(i)]$.
- Si nécessaire, échanger $A[i]$ avec le plus grand de ses deux enfants afin de préserver la propriété de tas (heap).
- Continuer ce processus de comparaison et d'échange vers le bas du tas jusqu'à ce que le sous-arbre enraciné en i soit un max-heap.

```
MAX-HEAPIFY(A, i, n)
    l = LEFT(i)
    r = RIGHT(i)

    if l <= n and A[l] > A[i]
        largest = l
    else
        largest = i

    if r <= n and A[r] > A[largest]
        largest = r

    if largest != i
        exchange A[i] with A[largest]
        MAX-HEAPIFY(A, largest, n)
```

Goal: Restore the max-heap property at node i
Time complexity: $Theta(heightofi) = O(\log n)$
Space: $O(\log n)$

Loop invariant: At the start of every iteration of the for loop, each node $i+1, i+2, \ldots, n$ is the root of a max-heap.

Maintenance:

- The children of node i are indexed higher than i, so by the loop invariant, they are both roots of max-heaps.
- Therefore, `MAX-HEAPIFY` makes node i a root of a max-heap (so $i, i+1, \ldots, n$ are all roots of max-heaps).
- Hence, the invariant remains true when i is decremented at the beginning of the next iteration.

4.2 Build-Heap

C'est bien toutes ces histoires de max-heap, de min-heap, de heap ect mais bon quand même au début il faudrait commencer par le constrruire et savoir le construite cette data structure !!!!!!!!!!

Call `MAX-HEAPIFY` on all internal nodes bottom-up. Though each call is $O(\log n)$, a tight analysis

gives $O(n)$ total.

Concept and idea:

The BUILD-MAX-HEAP algorithm takes an unordered array and transforms it into a valid max-heap.

It works by starting from the last non-leaf node and going backwards to the root. At each step, it applies MAX-HEAPIFY to ensure that the subtree rooted at that node satisfies the heap property. By doing this bottom-up, the algorithm efficiently builds a correct max-heap.

```
BUILD-MAX-HEAP(A, n)
    for i = n/2 downto 1
        MAX-HEAPIFY(A, i, n)
```

Goal: Build a max-heap from an unsorted array
Time complexity: $O(n)$
Space: $O(1)$

4.3 Heapsort

Concept and idea:

The HEAPSORT algorithm sorts an array by first building a max-heap, then repeatedly extracting the maximum element.

After building the max-heap, the largest element is always at the root. The algorithm swaps it with the last element of the heap, then "removes" it by reducing the heap size.

After each removal, MAX-HEAPIFY is applied to restore the heap property. This process is repeated until the entire array is sorted in increasing order.

```
HEAPSORT(A, n)
    BUILD-MAX-HEAP(A, n)

    for i = n downto 2
        exchange A[1] with A[i]
        MAX-HEAPIFY(A, 1, i - 1)
```

Goal: Sort an array using a max-heap
Time complexity: $O(n \log n)$
Space: $O(1)$

4.4 Heap extract max

Concept and idea:

The HEAP-EXTRACT-MAX operation removes the maximum element from a max-heap (the root) and restores the heap structure afterwards.

It works by first checking that the heap is not empty, then storing the root (maximum element). The last element of the heap is moved to the root position, and the heap size is reduced. Finally, `MAX-HEAPIFY` is called to restore the max-heap property.

```
HEAP-EXTRACT-MAX(A, n)
    if n < 1
        error "heap underflow"

    max = A[1]
    A[1] = A[n]
    n = n - 1

    MAX-HEAPIFY(A, 1, n)

    return max
```

Goal: Remove and return the maximum element of a heap
Time complexity: $O(\log n)$

Space: $O(1)$

4.5 Heap maximum in a max-heap

Concept and idea:

The `HEAP-MAXIMUM` operation simply returns the lagrest element in a max-heap.

Since the max-heap property guarantees that the maximum element is always stored at the root, this operation only needs to access the first element of the array.

```
HEAP-MAXIMUM(A)
    return A[1]
```

Goal: Return the maximum element of a max-heap

Time complexity: $O(1)$

Space: $O(1)$

4.6 Heap increase key

Concept and idea:

The `HEAP-INCREASE-KEY` operation updates the value of a node in a max-heap and restores the

heap property.

It assumes that the new key is not smaller than the current key. After updating the value, the algorithm moves the element upward in the tree, swapping it with its parent as long as it is larger. This process continues until the heap property is restored.

```
HEAP-INCREASE-KEY(A, i, key)
    if key < A[i]
        error "new key is smaller than current key"

    A[i] = key

    while i > 1 and A[PARENT(i)] < A[i]
        exchange A[i] and A[PARENT(i)]
        i = PARENT(i)
```

Goal: Update a key and restore the max-heap property
Time complexity: $O(\log n)$

Space: $O(1)$

4.7 Max heap insert

Concept and idea:

The `MAX-HEAP-INSERT` operation adds a new element into a max-heap while maintaining the heap property.

The idea is to first increase the heap size and place a temporary very small value (so it will not violate the heap structure). Then the algorithm uses `HEAP-INCREASE-KEY` to replace this value with the correct key and move the element upward until the heap property is restored.

```
MAX-HEAP-INSERT(A, key, n)
    n = n + 1
    A[n] = -8
    HEAP-INCREASE-KEY(A, n, key)
```

Goal: Insert a new element into a max-heap

Time complexity: $O(\log n)$

Space: $O(1)$

CHAPTER 5

Priority Queue

Concept and idea:

A **priority queue** maintains a dynamic set S of elements. Each element in the set has a **key**, which represents its priority (importance).

The structure supports efficient management of elements based on their keys, especially when priorities change over time.

Main operations:

- `INSERT(S, x)`: inserts element x into S
- `MAXIMUM(S)`: returns the element with the largest key
- `EXTRACT-MAX(S)`: removes and returns the element with the largest key
- `INCREASE-KEY(S, x, k)`: increases the key of element x to value k (where $k \geq$ current key of x)

Example application: scheduling jobs on a shared computer, where higher priority jobs are executed first.

Heaps are commonly used to efficiently implement priority queues.

Element

An **element** is an item stored in a priority queue. It consists of a value and an associated key that determines its priority.

Maximum element

The **maximum element** is the element in S with the largest key. It is always the element returned by a max-priority queue.

Priority queue operations

Beyond the basic operations, a priority queue relies on the idea that all updates maintain ordering by key, allowing fast access to the highest-priority element.

Application

Priority queues are widely used in scheduling systems (e.g., CPU job scheduling), where tasks with higher priority are executed first.

Property

`Maximum`	$O(1)$
`Extract-Max`	$O(\log n)$
`Insert`	$O(\log n)$
`Increase-Key`	$O(\log n)$

Heap implementation

BEN OUI FAUT QUE CA AIT DU SENS DE LE FAIRE APRES HEAP. DONC ON DEFINIT SYMPATIQUEMENT UNE PRIORITY QUEUE AVEC UN HEAP. A priority queue is efficiently implemented using a **heap**, which supports logarithmic-time insertion and extraction operations.

Une **priority queue** peut être implémentée à l'aide d'un **heap** pour gérer efficacement les éléments selon leur priorité.

On stocke tous les éléments dans un max-heap : le nœud au sommet (la racine) contient toujours l'élément avec la plus grande clé.

Grâce à cette structure :

- l'insertion se fait en ajoutant l'élément puis en remontant dans le heap,
- l'accès au maximum est direct (racine),
- la suppression du maximum se fait en remplaçant la racine par le dernier élément puis en réorganisant le heap.

Ainsi, le heap permet de réaliser une priority queue efficace, avec des opérations rapides en temps logarithmique.

(Dans le cas où on a une paire clé-valeur et pas juste ici les valeurs de priorité, alors on met dans le heap ces paires et on bouge les deux elements en meme temps mais on regarde roujours seulement la valeur de la key pour l'ordering)

CHAPTER 6

Stacks

Stack (pile)

A **stack** is a linear data structure that follows the LIFO principle (Last In, First Out). This means that the last element added is the first one to be removed.

LIFO principle

LIFO (Last In, First Out) means that elements are removed in the reverse order of insertion. The most recently inserted element is always at the top of the stack.

Top

The **top** is the position of the last inserted element in the stack. All insertions and deletions happen only at the top.

Push operation

`PUSH` adds an element to the top of the stack. After the operation, this element becomes the new top.

Pop operation

`POP` removes and returns the element at the top of the stack. The next element below becomes the new top.

Stack underflow

A **stack underflow** occurs when trying to pop an element from an empty stack. This is an invalid operation.

Stack overflow

A **stack overflow** occurs when trying to push an element into a full stack (if the size is bounded).

Applications

Stacks are used in function calls (call stack), expression evaluation, undo mechanisms, and depth-first search (DFS).

Stack implementation (how it is stored)

A stack is usually implemented using an array $S[1..n]$ and a variable `top`.
`top` keeps track of the index of the last inserted element:

- If `top = 0`, the stack is empty
- Elements are stored from $S[1]$ up to $S[top]$
- All operations (push and pop) modify only the `top`

Fixed size stack: The array size is fixed in advance. If the stack becomes full, no more elements can be added (stack overflow).
Dynamic size stack: The array can grow when it becomes full (usually by allocating a larger array and copying elements). This avoids overflow but may require resizing operations. This makes the stack very efficient, since no shifting of elements is needed. (PAS CE QUON A FAIT)

6.1 Stack empty

Concept and idea:

The `STACK-EMPTY` operation checks whether a stack contains any elements or not. It simply tests the position of the top pointer: if the stack has no elements, the top is equal to 0.

```
STACK-EMPTY(S)
    if S.top == 0
        return TRUE
    else
        return FALSE
```

Goal: Check whether a stack is empty
Time complexity: $O(1)$
Space: $O(1)$

6.2 Stack push

Concept and idea:

The `PUSH` operation adds an element to the top of the stack. It increases the `top` pointer and stores the new value at that position.

```
PUSH(S, x)
    S.top = S.top + 1
    S[S.top] = x
```

Goal: Insert an element into a stack

Time complexity: $O(1)$

Space: $O(1)$

6.3 Stack pop

Concept and idea:

The `POP` operation removes and returns the top element of the stack. It first checks if the stack is empty to avoid underflow, then decreases the `top` pointer.

```
POP(S)
    if STACK-EMPTY(S)
        error "underflow"
    else
        S.top = S.top - 1
        return S[S.top + 1]
```

Goal: Remove and return the top element of a stack

Time complexity: $O(1)$

Space: $O(1)$

CHAPTER 7

Queue

Queue (file)

A **queue** is a linear data structure that follows the FIFO principle (First In, First Out). This means that the first element added is the first one to be removed.

FIFO principle

FIFO (First In, First Out) means that elements are removed in the same order they were inserted. The oldest element in the structure is always the first to leave.

Front

The **front** is the position of the first element in the queue. Deletions (dequeue) always happen at the front.

Rear

The **rear** is the position where new elements are inserted. Insertions (enqueue) always happen at the rear.

Enqueue operation

`ENQUEUE` adds an element at the rear of the queue. After insertion, the new element becomes the last element.

Dequeue operation

`DEQUEUE` removes and returns the element at the front of the queue. The next element becomes the new front.

Queue underflow

A **queue underflow** occurs when trying to dequeue from an empty queue. This is an invalid operation.

Queue overflow

A **queue overflow** occurs when trying to enqueue into a full queue (if bounded).

Applications

Queues are used in scheduling systems, breadth-first search (BFS), printer queues, and buffering.

Queue implementation (how it is stored)

A queue is usually implemented using an array $Q[1..n]$ with two pointers: `front` and `rear`.

- `front` points to the first element to remove
- `rear` points to the position where the next element is inserted
- If `front > rear`, the queue is empty

Fixed size queue: The array has a fixed capacity. When full, no more elements can be inserted.
Dynamic size queue: The array can grow when needed (by resizing and copying elements), avoiding overflow but with occasional cost.
This structure is efficient because insertions and deletions do not require shifting elements.(MAIS C PAS CE QUON A FAIT LA C DEF EN PLUS)

Implementation using arrays: Q consists of elements $S[Q.head, \ldots, Q.tail - 1]$

- $Q.head$ points at the first element (the next element to be removed)
- $Q.tail$ points at the next location where a newly arrived element will be placed

7.1 ENQUEUE

Concept and idea:

The `ENQUEUE` operation inserts a new element at the rear of the queue. In a circular array implementation, it places the element at `Q.tail` and then updates the tail pointer, wrapping around if needed.

```
ENQUEUE(Q, x)
    Q[Q.tail] = x

    if Q.tail == Q.length
        Q.tail = 1
    else
        Q.tail = Q.tail + 1
```

Goal: Insert an element into a queue

Time complexity: $O(1)$

Space: $O(1)$

7.2 DEQUEUE

Concept and idea:

The `DEQUEUE` operation removes and returns the element at the front of the queue. In a circular array implementation, it reads the element at `Q.head` and then advances the head pointer, wrapping around if needed.

```
DEQUEUE(Q)
    x = Q[Q.head]

    if Q.head == Q.length
        Q.head = 1
    else
        Q.head = Q.head + 1

    return x
```

Goal: Remove and return the front element of a queue

Time complexity: $O(1)$

Space: $O(1)$

7.3 Stacks and Queues pros and cons

Positives	**Negatives**
Very efficient	Limited support: for example, no search
Natural operations	Implementations using arrays have a fixed capacity

CHAPTER 8

Linked List

Linked List

A **linked list** is a linear data structure where elements are stored in nodes. Each node contains a value and a pointer to the next node in the sequence. Unlike arrays, elements are not stored contiguously in memory.

Node

A **node** is the basic building block of a linked list. It contains:

- a **key/value** (the data)
- a **next pointer** pointing to the next node

Head

The **head** is the first node of the linked list. It is the entry point used to access the entire structure.

Tail

The **tail** is the last node of the linked list. Its next pointer is `NULL`, meaning the end of the list.

Next pointer

The **next pointer** links one node to the next node in the list. It defines the ordering of elements.

Insertion

Insertion in a linked list means creating a new node and updating pointers to include it in the list. It can be done at the head, tail, or in the middle depending on the case.

Deletion

Deletion removes a node from the list by changing pointers so that the previous node points to the next one.

Advantages

- Dynamic size (no fixed capacity)
- Efficient insertions and deletions (no shifting required)

Disadvantages

- No direct access to elements (no indexing)
- Extra memory usage due to pointers

Applications

Linked lists are used in dynamic memory management, graph adjacency lists, and implementing stacks/queues.

Representation:

A linked list is represented as:

$$\text{Head} \to node_1 \to node_2 \to \cdots \to \text{NULL}$$

8.1 Linked list search

Concept and idea:

The `LIST-SEARCH` operation looks for an element with a given key in a linked list.

It starts from the head of the list and traverses node by node until it either finds the key or reaches the end of the list.

```
LIST-SEARCH(L, k)
    x = L.head

    while x != NIL and x.key != k
        x = x.next

    return x
```

Goal: Find an element with key k in a linked list

Time complexity: $O(n)$

Space: $O(1)$

8.2 Linked list insert

Concept and idea:

The `LIST-INSERT` operation inserts a new node at the beginning of a doubly linked list.

It updates pointers so that the new node becomes the new head of the list, and properly connects it to the previous first node if it exists.

```
LIST-INSERT(L, x)
    x.next = L.head

    if L.head != NIL
        L.head.prev = x

    L.head = x
    x.prev = NIL
```

Goal: Insert a new element at the head of a linked list

Time complexity: $O(1)$

Space: $O(1)$

8.3 Linked list delete

Concept and idea:

The `LIST-DELETE` operation removes a given node x from a doubly linked list.

It updates the pointers of the neighboring nodes so that the list remains correctly connected after deletion. Depending on whether x is the head or not, the head pointer may also need to be updated.

```
LIST-DELETE(L, x)
    if x.prev != NIL
        x.prev.next = x.next
    else
        L.head = x.next

    if x.next != NIL
        x.next.prev = x.prev
```

Goal: Delete a node from a doubly linked list

Time complexity: $O(1)$

Space: $O(1)$

8.4 Simplified pseudocodes

```
LIST-INSERT'(L, x)
    x.next = L.nil.next
    L.nil.next.prev = x
    L.nil.next = x
    x.prev = L.nil
```

```
LIST-DELETE'(L, x)
    x.prev.next = x.next
    x.next.prev = x.prev
```

CHAPTER 9

Binary Search Tree

Binary Search Tree (BST)

A **binary search tree** is a binary tree where each node follows a specific ordering rule: all values in the left subtree are smaller than the node, and all values in the right subtree are larger.

Node

A **node** is the basic element of a BST. It contains:

- a **key/value**
- a **left pointer**
- a **right pointer**
- (optionally) a **parent pointer**

BST property

For every node in the tree:

- all keys in the left subtree are smaller
- all keys in the right subtree are larger

This property is applied recursively at every node.

Search

Searching in a BST means starting from the root and going left or right depending on comparisons with the current node's key.

Insertion

Insertion adds a new node by following the BST property until reaching a NULL position where the node is inserted.

Deletion

Deletion removes a node while preserving the BST property. It may require replacing the node with its successor or predecessor.

Minimum and Maximum

- Minimum: leftmost node in the tree
- Maximum: rightmost node in the tree

Applications

BSTs are used in searching, sorting, symbol tables, and implementing dynamic ordered sets.

Representation:

A BST is represented as:

$$\text{root} \rightarrow \text{left subtree (smaller)}, \text{right subtree (larger)}$$

BST property

For every node x: all keys in the left subtree $\leq x.key \leq$ all keys in the right subtree.

All basic operations run in $O(h)$ where h is the height. For a *random* BST, $\mathbb{E}[h] = O(\log n)$; in the worst case $h = n - 1$.

9.1 Tree search

Concept and idea:

The `TREE-SEARCH` operation looks for a node with key k in a binary search tree.

It starts from the root (or a given node) and uses the BST property: if the key is smaller, it goes left; otherwise, it goes right.

```
TREE-SEARCH(x, k)
    if x == NIL or k == x.key
        return x

    if k < x.key
        return TREE-SEARCH(x.left, k)
    else
        return TREE-SEARCH(x.right, k)
```

Goal: Find a node with key k in a BST

Time complexity: $O(h)$ (where h is the height of the tree)

Space: $O(h)$ (recursive call stack)

9.2 Tree minimum

Concept and idea:

The `TREE-MINIMUM` operation finds the smallest key in a binary search tree by always going to the left child until reaching a node with no left child.

```
TREE-MINIMUM(x)
    while x.left != NIL
        x = x.left
    return x
```

Goal: Find the minimum element in a BST

Time complexity: $O(h)$

Space: $O(1)$

9.3 Tree maximum

Concept and idea:

The `TREE-MAXIMUM` operation finds the largest key in a binary search tree by always going to the right child until reaching a node with no right child.

```
TREE-MAXIMUM(x)
    while x.right != NIL
        x = x.right
    return x
```

Goal: Find the maximum element in a BST

Time complexity: $O(h)$

Space: $O(1)$

9.4 Tree successor

Concept and idea:

The `TREE-SUCCESSOR` operation finds the next node in in-order traversal of a binary search tree.

If the node has a right subtree, the successor is simply the minimum element of that right subtree. Otherwise, the algorithm moves up using parent pointers until it finds a node that is a left child of its parent.

```
TREE-SUCCESSOR(x)

    if x.right != NIL
        return TREE-MINIMUM(x.right)

    y = x.p

    while y != NIL and x == y.right
        x = y
        y = y.p

    return y
```

Goal: Find the in-order successor of a node in a BST

Time complexity: $O(h)$

Space: $O(1)$

9.5 Inorder tree walk

Concept and idea:

The `INORDER-TREE-WALK` operation visits all nodes of a binary search tree in sorted order.

It follows the in-order traversal strategy: left subtree → node → right subtree.

```
INORDER-TREE-WALK(x)

    if x != NIL
        INORDER-TREE-WALK(x.left)
        print key[x]
        INORDER-TREE-WALK(x.right)
```

Goal: Traverse a BST in sorted (in-order) order

Time complexity: $O(n)$

Space: $O(h)$ (recursion stack)

9.6 Preorder tree walk

Concept and idea:

The `PREORDER-TREE-WALK` operation traverses a binary tree by visiting the root first, then the left subtree, and finally the right subtree.

This is useful when you want to process a node before its children.

```
PREORDER-TREE-WALK(x)

    if x != NIL
        print key[x]
        PREORDER-TREE-WALK(x.left)
        PREORDER-TREE-WALK(x.right)
```

Goal: Traverse a BST in preorder (root → left → right)

Time complexity: $O(n)$

Space: $O(h)$ (recursion stack)

9.7 Postorder tree walk

Concept and idea:

The `POSTORDER-TREE-WALK` operation traverses a binary tree by visiting the children first, and the root last.

It follows the order: left subtree → right subtree → node.

```
POSTORDER-TREE-WALK(x)

    if x != NIL
        POSTORDER-TREE-WALK(x.left)
        POSTORDER-TREE-WALK(x.right)
        print key[x]
```

Goal: Traverse a BST in postorder (left → right → root)

Time complexity: $O(n)$

Space: $O(h)$ (recursion stack)

9.8 Tree insert

Concept and idea:

The `TREE-INSERT` operation inserts a new node into a binary search tree while preserving the BST property.

It first searches for the correct position by moving down the tree, then attaches the new node as a leaf.

```
TREE-INSERT(T, z)

    y = NIL
    x = T.root

    while x != NIL
        y = x
        if z.key < x.key
            x = x.left
```

```
        else
            x = x.right

    z.p = y

    if y == NIL
        T.root = z

    elseif z.key < y.key
        y.left = z

    else
        y.right = z
```

Goal: Insert a node into a BST

Time complexity: $O(h)$

Space: $O(1)$

9.9 Tree delete

Concept and idea:

The `TREE-DELETE` operation removes a node from a binary search tree while preserving the BST property. It relies on a helper operation called `TRANSPLANT`, which replaces one subtree with another.

The `TRANSPLANT` method is used to replace one subtree with another while correctly updating parent pointers, allowing the tree structure to be modified safely without breaking the binary search tree properties.

The `TREE-DELETE` operation removes a node from a binary search tree while preserving the BST property. It handles all structural cases (no child, one child, or two children) and uses `TRANSPLANT` to correctly reconnect subtrees when needed.

There are three cases when deleting a node z:

- If z has no children, it is simply removed.
- If z has one child, that child takes z's position.
- If z has two children, we find its successor y and replace z with y.

```
TRANSPLANT(T, u, v)

    if u.p == NIL
        T.root = v

    elseif u == u.p.left
        u.p.left = v

```

```
    else
        u.p.right = v

    if v != NIL
        v.p = u.p
```

```
TREE-DELETE(T, z)

    if z.left == NIL
        TRANSPLANT(T, z, z.right)

    elseif z.right == NIL
        TRANSPLANT(T, z, z.left)

    else
        y = TREE-MINIMUM(z.right)

        if y.p != z
            TRANSPLANT(T, y, y.right)
            y.right = z.right
            y.right.p = y

        TRANSPLANT(T, z, y)
        y.left = z.left
        y.left.p = y
```

Goal: Delete a node from a BST while preserving its structure

Time complexity: $O(h)$

Space: $O(1)$

Warning

A BST is only efficient when *balanced*. For guaranteed $O(\log n)$, use a self-balancing variant such as a **Red-Black tree** (height $\leq 2\log_2(n+1)$).

CHAPTER 10

Data Structures Comparison

Structure	Search	Insert	Delete	Notes
Unsorted array	$O(n)$	$O(1)$	$O(n)$	simple
Sorted array	$O(\log n)$	$O(n)$	$O(n)$	binary search
Linked list	$O(n)$	$O(1)$	$O(1)^*$	*with pointer
BST	$O(h)$	$O(h)$	$O(h)$	$h = O(n)$ worst
Red-Black tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	balanced
Heap	$O(n)$	$O(\log n)$	$O(\log n)$	max/min in $O(1)$
Hash table	$O(1)^*$	$O(1)^*$	$O(1)^*$	*expected

Data Structure	**Properties**
Stacks	Last-In-First-Out (LIFO). Insertion and deletion in $O(1)$ time. Array implementation with fixed capacity.
Queues	First-In-First-Out (FIFO). Insertion and deletion in $O(1)$ time. Array implementation with fixed capacity.
Linked Lists	No fixed capacity. Insertion and deletion in $O(1)$ time (if position is known). Search takes $O(n)$ time.
Binary Search Trees	No fixed capacity. Supports insertion, deletion, search, minimum and maximum in $O(h)$ time, where h is the height of the tree.

Table 10.1: Summary of basic data structures and their complexities

CHAPTER 11

Dynamic programming

Dynamic Programming (DP)

Main idea:

- Remember calculations already made
- Saves enormous amounts of computation
- Allows to solve many optimization problems
- Always at least one question in Google Code Jam needs DP

Dynamic Programming (DP)

Dynamic Programming is a technique used to solve problems by breaking them into smaller subproblems and reusing already computed results instead of recomputing them.

Main idea

- Store results of subproblems (memoization or tabulation)
- Avoid recomputing the same values multiple times
- Transform exponential problems into polynomial time solutions

Overlapping subproblems

A problem has overlapping subproblems when the same subproblems are solved multiple times during recursion.

Optimal substructure

A problem has optimal substructure when the optimal solution can be built from optimal solutions of its subproblems.

Memoization (Top-down)

Memoization is a DP technique where results are stored during recursion to avoid recomputation.

Tabulation (Bottom-up)

Tabulation is a DP technique where solutions are built iteratively from the smallest subproblems to the final solution.

Applications

Dynamic Programming is used in optimization problems such as shortest paths, knapsack, sequence alignment, and many combinatorial problems.

Representation:

DP is generally represented as:

$$dp[i] = \text{optimal solution of subproblem } i$$

or more generally:

$$dp[state] = \text{best value achievable for a given state}$$

11.1 IMPORTANT CONCEPT BASICS

Two ways of Dynamic Programming

Dynamic Programming can be implemented in two main ways: top-down and bottom-up approaches.

Top-down (Memoization)

- Solve the problem recursively
- Store each computed result in a table
- Avoid recomputing already solved subproblems
- Memoization means "remembering previously computed results"

Bottom-up (Tabulation)

- Solve subproblems in increasing order of size
- Start from the smallest subproblems
- Use already computed results to build larger solutions

11.2 Exemples on fibonnaci

```
Bottom-Up-Fib(n)

1. Let r = [0 ... n] be a new array
2. r[0] \leftarrow 1
3. r[1] \leftarrow 1
4. for i = 2 to n
5.     r[i] \leftarrow r[i - 1] + r[i - 2]
6. return r[n]
```

```
Memoized-Fib(n)

1. Let r = [0 ... n] be a new array
2. for i = 0 to n
3.     r[i] \leftarrow -\infty
4. return Memoized-Fib-Aux(n, r)
```

```
Memoized-Fib-Aux(n, r)
```

```
2
3  1. if r[n] \geq 0
4  2.     return r[n]
5  3. if n = 0 or n = 1
6  4.     ans \leftarrow 1
7  5. else
8  6.     ans \leftarrow Memoized-Fib-Aux(n - 1, r) +
9                      Memoized-Fib-Aux(n - 2, r)
10 7. r[n] \leftarrow ans
11 8. return r[n]
```

11.3 Key element on designing DP

Optimal Substructure

Optimal substructure means that a solution to a problem is built by making a choice that leads to one or more subproblems, and the optimal solution is obtained by solving these subproblems optimally.

Overlapping Subproblems

A problem has overlapping subproblems when a naive recursive solution solves the same subproblems multiple times.

Top-down with Memoization

Solve the problem recursively while storing each computed result in a table to avoid recomputation.

Bottom-up Approach

Sort the subproblems by size and solve the smallest ones first. This ensures that when solving a subproblem, all required smaller subproblems have already been solved.

11.4 Rod Cutting algorithm

Concept and idea:

The rod cutting problem consists in cutting a rod of length n into smaller pieces in order to maximize the total profit, given a price table $p[i]$.

This dynamic programming solution uses memoization to avoid recomputing the same subproblems multiple times.

```
1 Memoized-Cut-Rod(p, n)
2
```

```
1. let r[0 .. n] be a new array
2. for i = 0 to n
3.     r[i] = -8
4. return Memoized-Cut-Rod-Aux(p, n, r)
```

Goal: Compute the maximum revenue obtainable by cutting a rod of size n

Time complexity: $O(n^2)$

Space: $O(n)$

—

```
Memoized-Cut-Rod-Aux(p, n, r)

1. if r[n] >= 0
2.     return r[n]

3. if n == 0
4.     q = 0
5. else
6.     q = -8
7.     for i = 1 to n
8.         q = max(q, p[i] + Memoized-Cut-Rod-Aux(p, n - i, r))

9. r[n] = q
10. return q
```

Goal: Compute the optimal revenue for a rod of length n using memoization

Time complexity: $O(n^2)$

Space: $O(n)$

—

What the algorithm does:

The algorithm tries all possible first cuts i, and recursively solves the remaining subproblem of size $n - i$. Memoization ensures that each subproblem is solved only once.

—

General idea:

- Keep the recursive structure of the problem
- Store results of subproblems (memoization)
- Reuse stored results instead of recomputing

—

Theorem (intuition):

If an optimal solution starts with a first cut at position i, then the remaining part of size $n - i$ must also be solved optimally.

Thus:

- optimal solution = optimal first cut + optimal solution of remaining subproblems

—

Problem solved:

This algorithm removes the exponential explosion of the naive recursive solution by avoiding repeated computations of the same rod sizes.

11.5 Rod Cutting algorithm bottom-up

Concept and idea:

The `BOTTOM-UP-CUT-ROD` algorithm solves the rod cutting problem using the bottom-up dynamic programming approach.

Instead of solving recursively like the memoized version, it starts from the smallest rod sizes and progressively builds solutions up to size n.

At each step:

- smaller subproblems are already solved
- the algorithm reuses these results directly from the table
- no recursive calls are needed

This avoids recursion overhead and guarantees that each subproblem is solved exactly once.

```
BOTTOM-UP-CUT-ROD(p, n)

let r[0..n] be a new array
r[0] = 0

for j = 1 to n
    q = -8
    for i = 1 to j
        q = max(q, p[i] + r[j - i])

    r[j] = q

return r[n]
```

Goal: Compute the maximum obtainable revenue for a rod of size n using bottom-up dynamic programming

Time complexity: $O(n^2)$

Space: $O(n)$

—

Difference with the memoized (top-down) approach:

- **Top-down:** starts from the final problem and recursively explores smaller subproblems only when needed.
- **Bottom-up:** starts from the smallest subproblems and iteratively builds all larger solutions until reaching the final answer.

- **Top-down:** uses recursion + memoization.
- **Bottom-up:** uses iteration + tabulation (array filling).

—

General idea:

- Sort subproblems by size
- Solve the smallest ones first
- Reuse already computed solutions to build larger ones

Recovering the Optimal Solution

The previous rod cutting algorithms only return the optimal profit. However, in many cases, we also want to know the actual cuts used to obtain this optimal solution.

Main idea

Each cell of the dynamic programming table corresponds to a decision: the position of the leftmost cut chosen for that rod size.

Approach

Store the decision associated with each subproblem in a separate table. This allows reconstruction of the sequence of cuts after computing the optimal profit.

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

let r[0..n] and s[0..n] be new arrays
r[0] = 0

for j = 1 to n
    q = -8
    for i = 1 to j
        if q < p[i] + r[j - i]
            q = p[i] + r[j - i]
            s[j] = i

    r[j] = q

return r and s
```

11.6 Matrix chain multiplication - find best separation

Concept and idea:

Matrix Chain Multiplication aims to determine the most efficient way to multiply a chain of matrices. The goal is not to change the order of multiplication, but to decide the optimal parenthesization that minimizes the number of scalar multiplications.

```
Input: A chain A1, A2, ..., An of n matrices
```

```
where Ai has dimension p[i-1] x p[i]

Output: Optimal parenthesization minimizing scalar multiplications
```

Theorem (Optimal substructure):

If an optimal solution splits the chain at position i:

$$(A_1 A_2 \cdots A_i)(A_{i+1} A_{i+2} \cdots A_n)$$

and if P_L and P_R are optimal solutions for the left and right parts, then combining them:

$$(P_L \cdot P_R)$$

is an optimal solution for the whole chain.

—

```
MATRIX-CHAIN-ORDER(p)

n = p.length - 1

let m[1..n, 1..n] and s[1..n, 1..n] be new tables

for i = 1 to n
    m[i, i] = 0

for l = 2 to n
    for i = 1 to n - l + 1
        j = i + l - 1
        m[i, j] = 8

        for k = i to j - 1
            q = m[i, k] + m[k + 1, j] + p[i - 1] * p[k] * p[j]

            if q < m[i, j]
                m[i, j] = q
                s[i, j] = k

return m and s
```

Goal: Find the optimal parenthesization minimizing scalar multiplications

Time complexity: $O(n^3)$

Space: $O(n^2)$

Concept and idea:

The `PRINT-OPTIMAL-PARENS` procedure reconstructs the optimal parenthesization of a matrix chain using the table s computed by the dynamic programming algorithm.

It recursively splits the chain at the optimal position and prints parentheses accordingly.

```
Print-Optimal-Parens(s, i, j)

1 if i == j
2     print "Ai"
3 else print "("
4     Print-Optimal-Parens(s, i, s[i, j])
5     Print-Optimal-Parens(s, s[i, j] + 1, j)
6     print ")"
```

Goal: Reconstruct and print the optimal parenthesization of a matrix chain

Time complexity: $O(n)$ (visits each split once)

Space: $O(h)$ (recursion stack, where h is recursion depth)

11.7 Longest common subsequence

Concept and idea:

The Longest Common Subsequence (LCS) problem consists in finding the longest sequence that appears in the same order (not necessarily consecutively) in two given sequences X and Y.

The idea is to compare characters step by step and build an optimal solution using dynamic programming.

```
INPUT: sequences X = x1, ..., xm and Y = y1, ..., yn
OUTPUT: longest common subsequence
```

Intuition:

We compare the sequences from the end:

- If the characters are equal, they are part of the LCS.
- If they are different, we move left in one of the sequences depending on which choice gives the best result.

—

Theorem (optimal substructure):

Let $Z = z_1, z_2, ..., z_k$ be an LCS of X_i and Y_j.

- If $x_i = y_j$, then $z_k = x_i = y_j$ and Z_{k-1} is an LCS of X_{i-1} and Y_{j-1}.
- If $x_i \neq y_j$, then:
  - either Z is an LCS of X_{i-1} and Y_j
  - or Z is an LCS of X_i and Y_{j-1}

This means that the optimal solution is built from optimal solutions of smaller subproblems.

—

```
LCS-LENGTH(X, Y, m, n)

let b[1..m, 1..n] and c[0..m, 0..n] be new tables

for i = 1 to m
    c[i, 0] = 0

for j = 0 to n
    c[0, j] = 0

for i = 1 to m
    for j = 1 to n
        if x_i == y_j
            c[i, j] = c[i - 1, j - 1] + 1
            b[i, j] = "<-"
        else if c[i - 1, j] >= c[i, j - 1]
            c[i, j] = c[i - 1, j]
            b[i, j] = "fleche en haut"
        else
            c[i, j] = c[i, j - 1]
            b[i, j] = "<-"

return c and b
```

—

```
PRINT-LCS(b, X, i, j)

if i == 0 or j == 0
    return

if b[i, j] == "fleche en haut a gauche"
    PRINT-LCS(b, X, i - 1, j - 1)
    print x_i

elseif b[i, j] == "fleche en haut"
    PRINT-LCS(b, X, i - 1, j)

else
    PRINT-LCS(b, X, i, j - 1)
```

—

Goal: Find the longest subsequence common to both sequences

Time complexity: $O(mn)$

Space: $O(mn)$

11.8 Bottom-up optimal BST

Concept and idea:

The Optimal Binary Search Tree (OBST) problem consists in building a BST from sorted keys such that the expected search cost is minimized, given probabilities of successful searches p_i and unsuccessful searches q_i.

The goal is not just to keep the tree valid, but to minimize the weighted search cost.

```
OPTIMAL-BST(p, q, n)

let e[1..n + 1, 0..n], w[1..n + 1, 0..n], and root[1..n, 1..n]
    be new tables

for i = 1 to n + 1
    e[i, i - 1] = 0
    w[i, i - 1] = 0

for l = 1 to n
    for i = 1 to n - l + 1
        j = i + l - 1
        e[i, j] = 8
        w[i, j] = w[i, j - 1] + p_j

        for r = i to j
            t = e[i, r - 1] + e[r + 1, j] + w[i, j]

            if t < e[i, j]
                e[i, j] = t
                root[i, j] = r

return e and root

```

Goal: Construct a BST with minimum expected search cost

Time complexity: $O(n^3)$

Space: $O(n^2)$

CHAPTER 12

Graphs

Graph (Graphe)

A **graph** is a data structure composed of a set of vertices (nodes) and edges (connections) between them. It is used to model relationships between objects.

Vertex (Node)

A **vertex** is a fundamental unit of a graph. It represents an object or entity in the graph.

Edge

An **edge** is a connection between two vertices. It can be directed or undirected, and may have a weight.

Directed Graph (Digraph)

A **directed graph** is a graph where edges have a direction, going from one vertex to another.

Undirected Graph

An **undirected graph** is a graph where edges have no direction; the relationship is symmetric.

Weighted Graph

A **weighted graph** is a graph where each edge has an associated cost or weight.

Adjacent vertices

Two vertices are **adjacent** if they are connected by an edge.

Degree

The **degree** of a vertex is the number of edges connected to it. In a directed graph, we distinguish in-degree and out-degree.

Path

A **path** is a sequence of vertices connected by edges.

Cycle

A **cycle** is a path that starts and ends at the same vertex.

Connected Graph

An undirected graph is **connected** if there is a path between every pair of vertices.

Strongly Connected Graph

A **directed graph** is strongly connected if for every pair of vertices u and v, there exists a path from u to v and a path from v to u.

Weakly Connected Graph

A **directed graph** is weakly connected if, after ignoring the direction of edges, the resulting undirected graph is connected.

Connected Components

A **connected component** is a maximal set of vertices such that every pair is connected by a path.

Tree (Graph Theory)

A **tree** is a connected acyclic graph.

Directed Acyclic Graph (DAG)

A **DAG** is a directed graph with no cycles.

Representation:

A graph is written as:

$$G = (V, E)$$

where:

- V is the set of vertices
- E is the set of edges

Weighted Vertex

A **weighted vertex** is a vertex that has an associated value (weight), which can represent cost, priority, distance, or any other metric.

Weighted Edge

A **weighted edge** is an edge that has an associated weight. This weight represents a cost, distance, time, or any numerical value assigned to the connection between two vertices.

Adjacency List Representation

An **adjacency list** represents a graph by storing, for each vertex, a list of all vertices directly connected to it.

Idea:

- Each vertex stores its neighbors
- Efficient for sparse graphs
- Space complexity: $O(V + E)$

Example:

$$A \to \{B, C\}, \quad B \to \{A, D\}$$

Adjacency Matrix Representation

An **adjacency matrix** represents a graph using a 2D matrix M where:

- $M[i][j] = 1$ (or weight) if there is an edge from i to j
- $M[i][j] = 0$ (or ∞) otherwise

Idea:

- Fast edge lookup: $O(1)$
- Uses $O(V^2)$ space
- Better for dense graphs

Example:
For a graph with vertices $\{A, B, C\}$ and edges $A \to B$, $B \to C$, the adjacency matrix is:

	A	B	C
A	0	1	0
B	0	0	1
C	0	0	0

	Adjacency List	**Adjacency Matrix**
Space	$\Theta(V + E)$	$\Theta(V^2)$
List adjacent to u	$\Theta(\deg(u))$	$\Theta(V)$
Test if $(u, v) \in E$	$O(\deg(u))$	$\Theta(1)$

Cycle Detection Lemma

A directed graph G is **acyclic** if and only if a Depth-First Search (DFS) of G produces no back edges.

Strongly Connected Component (SCC)

A **strongly connected component** (SCC) of a directed graph $G = (V, E)$ is a **maximal set of vertices** $C \subseteq V$ such that for every pair of vertices $u, v \in C$, there exists a path from u to v and a path from v to u.

Component Graph (SCC Graph)

For a directed graph $G = (V, E)$, its **component graph** $G_{SCC} = (V_{SCC}, E_{SCC})$ is defined as follows:

- V_{SCC} contains one vertex for each strongly connected component of G
- E_{SCC} contains an edge between two SCCs if there exists at least one edge in G connecting a vertex in the first SCC to a vertex in the second SCC

Intuition: each SCC is contracted into a single "super-node", and edges between SCCs define how these components are connected.

Lemma: The SCC Graph is a DAG (Directed acyclic graph)

For any directed graph $G = (V, E)$, the component graph G_{SCC} (formed by contracting each strongly connected component into a single vertex) is a directed acyclic graph (DAG).

Intuition: if there were a cycle between SCCs, then all vertices in those components would be mutually reachable, meaning they would actually belong to the same strongly connected component. This contradicts the definition of SCCs being maximal.

12.1 Breadth-First Search

Concept and idea:

The BFS (Breadth-First Search) algorithm explores a graph level by level starting from a source vertex s. It computes the shortest path (in number of edges) from s to every reachable vertex in the graph.

The idea is that vertices closer to the source are processed first using a queue, ensuring that the first time we reach a vertex, it is via the shortest path.

```
BFS(V, E, s)

for each u apparient V - {s}
    u.d = 8

s.d = 0
Q = ensemble vide
ENQUEUE(Q, s)

while Q != ensemble vide
    u = DEQUEUE(Q)

    for each v apparient G.Adj[u]
        if v.d == 8
            v.d = u.d + 1
            ENQUEUE(Q, v)
```

Goal: Compute the shortest distance (in number of edges) from source s to every vertex $v \in V$

Time complexity: $O(V + E)$

Space: $O(V)$ **Idea of correctness:**

If a vertex v had a distance greater than the true shortest path from s to v, this would contradict the BFS exploration order.

Since BFS explores vertices in increasing order of distance from the source (level by level using a queue), the first time a vertex is discovered is always via a shortest path.

—

Runtime analysis: $\mathcal{O}(V + E)$

- $\mathcal{O}(V)$: each vertex is enqueued at most once
- $\mathcal{O}(E)$: each edge is examined at most once (or twice for undirected graphs)

—

Additional notes:

- BFS may not reach all vertices if the graph is not connected
- We can reconstruct the shortest path tree by storing the predecessor (the edge that first discovered each vertex)

12.2 Depth-First Search

Concept and idea:

Depth-First Search (DFS) explores a graph by going as deep as possible along each branch before backtracking. It starts from a vertex, explores one path completely, then backtracks and continues with other unexplored paths.

Unlike BFS, which explores vertices level by level, DFS explores along depth first.

The algorithm assigns two timestamps to each vertex:

- discovery time $v.d$ when the vertex is first visited
- finishing time $v.f$ when exploration of the vertex is complete

```
DFS(G)

for each u appartient G.V
    u.color = WHITE

time = 0

for each u appartent G.V
    if u.color == WHITE
        DFS-VISIT(G, u)
```

```
DFS-VISIT(G, u)

time = time + 1
u.d = time
u.color = GRAY

for each v appartient G.Adj[u]
    if v.color == WHITE
        DFS-VISIT(v)

u.color = BLACK
time = time + 1
u.f = time
```

Goal: Explore all vertices and edges of the graph while computing discovery and finishing times

Time complexity: $O(V + E)$

Space: $O(V)$ (recursion stack)

DFS Vertex Colors

As Depth-First Search (DFS) progresses, each vertex is assigned a color to track its exploration state:

- **WHITE**: the vertex is undiscovered
- **GRAY**: the vertex is discovered but not fully explored (DFS is still processing its neighbors)
- **BLACK**: the vertex is finished (all reachable vertices from it have been explored)

These colors help DFS manage traversal order and avoid revisiting vertices.

12.3 Classification of edges

Tree Edge

A **tree edge** is an edge discovered while exploring a new vertex during DFS. It becomes part of the depth-first forest.

Back Edge

A **back edge** is an edge (u, v) where v is an ancestor of u in the DFS tree. It points back toward an already discovered ancestor.

Forward Edge

A **forward edge** is an edge (u, v) where v is a descendant of u in the DFS tree, but (u, v) is not a tree edge.

Cross Edge

A **cross edge** is any edge that is neither a tree edge, a back edge, nor a forward edge. It typically connects vertices belonging to different DFS branches.

12.4 Parentheses theorem

Parenthesis Theorem (DFS)

For any two vertices u and v in a depth-first search, exactly one of the following cases holds:

1. $$u.d < u.f < v.d < v.f \quad \text{or} \quad v.d < v.f < u.d < u.f$$
   Neither u nor v is a descendant of the other.
2. $$u.d < v.d < v.f < u.f$$
   Vertex v is a descendant of u.
3. $$v.d < u.d < u.f < v.f$$
   Vertex u is a descendant of v.

In other words, DFS discovery and finishing times form properly nested intervals: descendant vertices have intervals entirely contained within the intervals of their ancestors.

Explication simple : ce théorème permet de déterminer la relation entre deux sommets uniquement en regardant leurs temps de découverte (d) et de fin (f). Il sert notamment à savoir rapidement si un sommet est descendant d'un autre dans l'arbre DFS, sans avoir à parcourir de nouveau le graphe.

12.5 White Path Theorem

White-Path Theorem

A vertex v is a descendant of a vertex u in the DFS forest if and only if, at the time $u.d$, there exists a path from u to v consisting entirely of WHITE vertices (except u, which has just been colored GRAY).

Explication simple : ce théorème permet de savoir quels sommets deviendront descendants de u dans l'arbre DFS. Au moment où DFS découvre u, tous les sommets encore atteignables par un chemin composé uniquement de sommets non visités (WHITE) seront explorés à partir de u et feront donc partie de son sous-arbre DFS. Ce théorème est souvent utilisé pour démontrer formellement pourquoi DFS construit correctement ses arbres de parcours.

12.6 Topological sort

Concept and idea:

Topological Sort produces a linear ordering of the vertices of a Directed Acyclic Graph (DAG)

such that for every edge (u, v), vertex u appears before vertex v in the ordering.

The algorithm uses DFS and relies on finishing times. Vertices that finish later are placed earlier in the topological ordering.

```
TOPOLOGICAL-SORT(G)

1. Call DFS(G) to compute finishing times v.f for all v appartenant a G.V
2. Output vertices in order of decreasing finishing times
```

Goal: Compute a valid topological ordering of a DAG

Time complexity: $O(V + E)$

Space: $O(V)$

—

Implementation note:

Instead of sorting vertices by their finishing times:

- Output each vertex when it finishes and reverse the resulting list.
- Or insert each finished vertex at the front of a linked list.

When DFS terminates, the linked list contains the vertices in topologically sorted order.

—

Correctness idea:

We must show that for every edge $(u, v) \in E$:

$$v.f < u.f$$

When DFS explores (u, v):

- u is necessarily **GRAY**.
- v cannot be **GRAY**, because then v would be an ancestor of u, creating a back edge. By the cycle detection lemma, the graph would not be acyclic.
- If v is **WHITE**, then v becomes a descendant of u. By the Parenthesis Theorem:
$$u.d < v.d < v.f < u.f$$
- If v is **BLACK**, then v has already finished. Since we are still exploring from u, we have:
$$v.f < u.f$$

Therefore, for every edge (u, v):

$$v.f < u.f$$

which means that ordering vertices by decreasing finishing times guarantees that u appears before v.

12.7 SCC and magic algorithm

Strongly Connected Component (SCC)

A **strongly connected component** (SCC) of a directed graph $G = (V, E)$ is a **maximal set of vertices** $C \subseteq V$ such that for every pair of vertices $u, v \in C$, there exists a path from u to v and a path from v to u.

Concept and idea:

The SCC (Strongly Connected Components) algorithm decomposes a directed graph into its strongly connected components.

The idea is to use two DFS passes:

- First DFS computes finishing times.
- Transpose the graph (reverse all edges).
- Second DFS explores vertices in decreasing order of finishing times to identify SCCs.

```
SCC(G)

1. Call DFS(G) to compute finishing times u.f for all u
2. Compute G^T
3. Call DFS(G^T), considering vertices in order of decreasing u.f
4. Output each tree of the DFS forest as one SCC
```

Transpose graph:

$G^T = (V, E^T)$ where:

$$E^T = \{(u, v) : (v, u) \in E\}$$

In other words, all edges of G are reversed.

—

Observations:

- G^T can be built in $O(V + E)$ time using adjacency lists
- G and G^T have exactly the same strongly connected components

—

Goal: Decompose the graph into all SCCs

Time complexity: $O(V + E)$

Space: $O(V + E)$

CHAPTER 13

Flow networks

Flow network

A directed graph $G = (V, E)$ with:

- a **source** s and a **sink** t,
- a **capacity function** $c : V \times V \to \mathbb{R}_{\geq 0}$ (0 for non-edges),
- no antiparallel edges (replace (u, v) and (v, u) by splitting with an intermediate node).

A **flow** f must satisfy capacity ($0 \leq f(u, v) \leq c(u, v)$) and conservation ($\sum_v f(v, u) = \sum_v f(u, v)$ for all $u \neq s, t$).

The **value** of a flow is $|f| = \sum_v f(s, v) - \sum_v f(v, s)$.

Flow Network

A **flow network** is a directed graph $G = (V, E)$ where each edge has a capacity $c(u, v) \geq 0$. It contains:

- a **source** s
- a **sink** t

Flow is sent from s to t while respecting capacities.

Flow

A **flow** is a function $f(u, v)$ that represents how much "stuff" is sent along an edge, satisfying:

- $0 \leq f(u, v) \leq c(u, v)$ (capacity constraint)
- Flow conservation: incoming flow = outgoing flow for all vertices except s and t

Capacity

The **capacity** $c(u, v)$ is the maximum amount of flow allowed on edge (u, v).

Residual Capacity

The **residual capacity** represents how much additional flow can still be pushed through an edge:

$$c_f(u, v) = c(u, v) - f(u, v)$$

It also includes backward edges to allow flow cancellation.

Residual Graph

The **residual graph** G_f contains all edges with positive residual capacity. It represents all possible ways to increase or adjust the current flow.

Flow Value

The **value of a flow** is the total amount leaving the source:

$$|f| = \sum_{v \in V} f(s, v)$$

Max Flow Problem

The goal is to compute a flow of maximum value from s to t while respecting all capacity constraints.

Key Ideas to Know

- Flow conservation at intermediate vertices
- Capacity constraints on edges
- Residual graph allows augmentation
- Augmenting path increases total flow

Applications

Flow networks are used in:

- network routing
- bipartite matching
- scheduling
- transportation problems

Flow Constraints

Capacity constraint: For all $u, v \in V$,

$$0 \leq f(u, v) \leq c(u, v)$$

Flow conservation: For all $u \in V \setminus \{s, t\}$,

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

Interpretation:

- The amount of flow entering a node equals the amount of flow leaving it (except for source and sink)
- No flow is created or destroyed inside the network

Cut in Flow Networks

A **cut** in a flow network $G = (V, E)$ is a partition of the vertices into two disjoint sets S and $T = V \setminus S$ such that:

- $s \in S$ (the source is in S)
- $t \in T$ (the sink is in T)

Idea: a cut separates the graph into a "left side" (containing the source) and a "right side" (containing the sink). **Note:** Note that this equals the value of the flow; it's always the case! (talking about the net across flow)

Net Flow Across a Cut

The **net flow** across a cut (S, T) is defined as:

$$f(S,T) = \sum_{u \in S, v \in T} f(u,v) - \sum_{u \in S, v \in T} f(v,u)$$

Interpretation:

- The first term represents the flow leaving S toward T
- The second term represents the flow coming back from T to S

Idea: it measures how much flow actually crosses from the source side to the sink side of the cut.

Explication simple : une coupe (cut) consiste à "couper" le graphe en deux parties : une partie qui contient la source s, et une autre qui contient le puits t. Cela permet de mesurer combien de flux peut passer d'un côté à l'autre du réseau.

Flow Conservation Theorem

For any cut (S, T) in a flow network, the value of the flow satisfies:

$$|f| = f(S,T)$$

Idea: the total flow sent from the source is exactly equal to the net flow crossing any cut separating the source from the sink.

Capacity of a Cut

The **capacity** of a cut (S, T) is defined as:

$$c(S,T) = \sum_{u \in S, v \in T} c(u,v)$$

Idea: it represents the total maximum amount of flow that can cross from the source side S to the sink side T.

Max-Flow Min-Cut Theorem

The **maximum flow value** in a flow network is equal to the **minimum capacity of any cut** separating the source from the sink:

$$\max |f| = \min_{(S,T)} c(S,T)$$

IN OTHER WORDS : MAX-FLOW = MIN-CUT
Idea: the best possible flow you can send from s to t is exactly limited by the smallest "bottleneck" cut in the network.

Max-Flow Min-Cut Theorem (Equivalent Statements)

Let $G = (V, E)$ be a flow network with source s, sink t, capacities c, and a flow f. The following statements are equivalent:

1. f is a maximum flow
2. The residual graph G_f has no augmenting path
3. There exists a cut (S, T) such that $|f| = c(S, T)$, and this cut is a minimum cut

Idea: all three viewpoints describe the same optimal situation: no more flow can be pushed, and the flow is exactly limited by the smallest bottleneck in the network.

13.1 Maximum flow - Ford Fulkerson algorithm

Concept and idea:

The Maximum Flow problem consists in sending the maximum possible amount of flow from a source s to a sink t in a flow network, while respecting edge capacities and flow conservation.

The Ford-Fulkerson method solves this by repeatedly finding paths where additional flow can still be pushed (called augmenting paths), and increasing the flow along these paths until no more improvements are possible.

```
Ford-Fulkerson-Method(G, s, t)

1. Initialize flow f to 0
2. while there exists an augmenting path p in the residual network G_f
3.     augment flow f along p
4. return f
```

Basic idea:

- As long as there exists a path from s to t with available capacity on every edge
- Send additional flow along this path
- Update the residual network
- Repeat until no augmenting path exists

Goal: Compute the maximum possible flow from source s to sink t

Key insight: Each augmenting path increases the total flow, and the algorithm stops when no such path exists

Time complexity: Depends on the choice of augmenting paths (can be unbounded in general)
Time complexity: $O(E \cdot |f^*|)$ (for integer capacities, where $|f^*|$ is the value of the maximum flow)
Space complexity: $O(V + E)$ (to store the residual graph)

CHAPTER 14

Disjoint-set data structures

Disjoint-Set Data Structure (Union-Find)

A **disjoint-set data structure** maintains a collection

$$S = \{S_1, S_2, \ldots, S_k\}$$

of pairwise disjoint dynamic sets (sets that change over time).
Each set is identified by a **representative** element, which is a member of the set. The representative must be consistent: if we ask for it multiple times without modifying the set, it must return the same value.

Make-Set

Make-Set(x) creates a new set containing only x:

$$S = \{\{x\}\}$$

Idea: each element starts in its own singleton set.

Find-Set

Find-Set(x) returns the representative of the set containing x.
Idea: it identifies which component/set an element belongs to.

Union

Union(x, y) merges the two sets containing x and y:

$$S_x \cup S_y$$

Idea: combines two disjoint sets into one single set, destroying the original two sets.
The representative of the new set is usually one of the two previous representatives.

Key Properties

- Sets are always disjoint
- Each element belongs to exactly one set
- Structure evolves dynamically over time

Applications

Disjoint-set structures are used in:

- Kruskal's algorithm (Minimum Spanning Tree)
- Connected components in graphs
- Cycle detection in undirected graphs

Operations Summary

- Make-Set(x): create a new singleton set $\{x\}$
- Find-Set(x): return representative of x
- Union(x, y): merge the sets containing x and y

Path Compression

During **Find-Set**, each visited node is directly linked to the representative (root).
Idea: flatten the structure of the tree so that future queries become much faster.

Union by Rank / Size

When performing **Union**, attach the smaller tree under the root of the larger tree (by rank or size).
Idea: keep trees as shallow as possible to speed up future operations.

Representation + Intuition

Each element stores:

- a parent pointer (itself if it is a root)
- optionally a rank or size

Intuition: the structure is a forest of trees where each tree represents a set.
Goal: efficiently support merging sets and checking whether two elements belong to the same set.

14.1 Connected components algorithm

Concept and idea:

The Connected Components algorithm finds all connected components of an undirected graph by using the Union-Find (Disjoint Set) data structure to group vertices that are reachable from each other.

Initially, each vertex starts in its own component, and edges are used to progressively merge these components.

```
CONNECTED-COMPONENTS(G)

for each vertex v apartenat a G.V
    MAKE-SET(v)

for each edge (u, v) appartenant a G.E
    if FIND-SET(u) != FIND-SET(v)
        UNION(u, v)
```

Basic idea:

- Start with each vertex in its own set
- Process edges one by one

- Merge sets when an edge connects two different components
- At the end, each set represents a connected component

Goal: Compute all connected components of an undirected graph

Time complexity: $O(E \cdot \alpha(V))$

Space complexity: $O(V)$

14.2 Disjoint-Set via Linked List Representation

Disjoint-Set via Linked List Representation

In this representation, each set is stored as a **single linked list**.
Each set has a **set object** containing:

- a pointer to the **head** of the list (the representative)
- a pointer to the **tail** of the list

Each element in the list stores:

- the value (set member)
- a pointer to the set object it belongs to
- a pointer to the next element in the list

Operations:

- **Make-Set(x):** creates a new list containing only x in $O(1)$ time
- **Find(x):** follows the pointer to the set object, then returns the head (representative), in $O(1)$ time
- **Union(x, y):** merges the list of y into the list of x

Union Operation (Linked List)

There are two main ways to implement **Union**:

1. Simple append strategy

- Append the list of y to the end of the list of x
- Use the tail pointer of x to access the end in $O(1)$ time
- Update the set pointer for every element in y's list

Problem: if a large list is appended to a small one, updating pointers can take a lot of time.

2. Weighted-union heuristic

- Always append the smaller list to the larger list
- Break ties arbitrarily

Idea: reduce the number of relabeling operations by avoiding expensive merges.

Weighted-Union Theorem

With the weighted-union heuristic, any sequence of m operations on n elements takes:

$$O(m + n \log n)$$

Idea (simple explanation): each time an element is moved to another list, it goes into a strictly larger set. So it can only be moved a limited number of times, which keeps the total cost low.
Goal: guarantee that even many union operations remain efficient overall.

14.3 Forest of trees

Disjoint-Set via Forest of Trees

In this representation, each set is stored as a **tree**.

- Each tree corresponds to one set
- The **root** of the tree is the representative of the set
- Each node stores only a pointer to its **parent**

Idea: instead of storing lists, we represent sets as trees, which makes unions more efficient.
Explication simple : ici, chaque ensemble est vu comme un arbre. Tous les éléments "remontent" vers une racine commune qui représente leur groupe.

Operations

- **Make-Set(x):** creates a single-node tree where x is its own root
- **Find(x):** follows parent pointers until reaching the root (representative)
- **Union(x, y):** attaches the root of one tree under the root of another

Idea: operations are simple pointer manipulations on tree roots.
Explication simple : Make-Set crée un groupe avec un seul élément, Find remonte jusqu'au chef du groupe, et Union fusionne deux groupes en reliant leurs racines.

Intuition and Purpose

This structure is used to efficiently manage dynamic connectivity, i.e., groups of elements that change over time.
Why it is useful:

- Much faster than linked lists for large datasets
- Easily merges groups of elements
- Becomes extremely efficient with optimizations (path compression + union by rank)

Explication simple : cela permet de savoir rapidement si deux éléments sont dans le même groupe, même si les groupes changent tout le temps.

14.4 Global idea (Union-Find / Disjoint Set) algorithms basics

Global idea (Union-Find / Disjoint Set):

The Union-Find structure maintains a collection of disjoint sets and supports two main operations:

- quickly finding which set an element belongs to
- efficiently merging two sets

It represents each set as a tree where each node points to its parent, and the root is the representative.

14.4.1 MAKE-SET

```
MAKE-SET(x)

1. x.p = x
2. x.rank = 0
```

What it does: creates a new set containing only x.

Purpose: initialize the structure so every element starts in its own group.

Idea: each element is initially its own root.

Time complexity: $O(1)$

Data structure state: single-node tree.

14.4.2 FIND-SET

```
FIND-SET(x)

1. if x != x.p
2.     x.p = FIND-SET(x.p)
3. return x.p
```

What it does: returns the representative (root) of the set containing x.

Purpose: identify which group an element belongs to.

Idea: climb parent pointers to the root + compress the path to speed up future queries.

Time complexity: amortized $O(\alpha(n))$

Data structure state: trees become flatter over time (path compression).

14.4.3 UNION

```
UNION(x, y)

1. LINK(FIND-SET(x), FIND-SET(y))
```

What it does: merges the two sets containing x and y.

Purpose: combine two separate groups into one.

Idea: find roots first, then attach one tree to the other.

Time complexity: amortized $O(\alpha(n))$

Data structure state: two trees become one.

14.4.4 LINK

```
LINK(x, y)

1. if x.rank > y.rank
2.     y.p = x
3. else x.p = y
4.     if x.rank == y.rank
5.         y.rank = y.rank + 1
```

What it does: attaches one root under another.

Purpose: merge trees while keeping them as shallow as possible.

Idea: union by rank prevents tall trees by always attaching the smaller height tree under the larger one.

Time complexity: $O(1)$

Data structure state: balanced forest structure is maintained.

14.5 CONNECTED COMPONENTS WITH THIS IMPLEMENTATION

NEW TIME COMPLEXITY

Total running time if implemented as linked list with weighted-union heuristic O(V log V + E)

CHAPTER 15

Minimum spanning trees

Spanning Tree

A **spanning tree** of a connected graph $G = (V, E)$ is a subset of edges $T \subseteq E$ such that:

- T is **acyclic** (contains no cycle)
- T is **spanning** (it connects all vertices in V)

Equivalent property: a spanning tree contains exactly $|V| - 1$ edges.

—

Spanning Tree (Intuition)

A spanning tree is a way to connect all vertices of a graph using the minimum number of edges, without forming any cycle.
Idea: it keeps only the "essential" connections needed to maintain connectivity.
En mots simples : un spanning tree est une version simplifiée du graphe où tout est encore connecté, mais sans aucun cycle (aucun "chemin en boucle").

—

Properties of a Spanning Tree

Let T be a spanning tree of a connected graph G:

- T connects all vertices
- T has no cycles
- T has exactly $|V| - 1$ edges
- There is a unique simple path between any two vertices

Idea: a spanning tree is minimally connected and maximally acyclic.

—

Why Spanning Trees are Important

Spanning trees are used to:

- reduce network cost (fewer edges)
- build efficient communication structures
- design minimum-cost networks (MST problems)
- simplify graph structures while preserving connectivity

En mots simples : on garde seulement les connexions essentielles pour que tout reste relié, mais en utilisant le minimum d'arêtes possible.

—

Spanning Tree vs General Graph

- A general graph may contain cycles and many redundant edges
- A spanning tree removes all redundancy
- It preserves connectivity but removes all cycles

Idea: spanning tree = skeleton of the graph.

Cut Property

Consider a cut $(S, V \setminus S)$ and let

- T be a tree on S which is part of a MST
- e be a crossing edge of minimum weight

Then there exists an MST of G containing e and T.
Explication simple : cette propriété dit du coup alors que si on coupe le graphe en deux parties, l'arête la moins chère (donc celle qui a la plus petite valuer ici comme ca) qui relie ces deux parties peut toujours faire partie d'un arbre couvrant minimal (MST).

15.1 Prim's algorithm

Concept and idea:

Prim's algorithm finds a **Minimum Spanning Tree (MST)** of a weighted graph by starting from a root vertex and repeatedly adding the cheapest edge that connects a visited vertex to an unvisited one.

It grows a tree step by step, always choosing the locally optimal (minimum weight) edge.

```
PRIM(G, w, r)

Q = ensemble vide

for each u appartien a G.V
    u.key = 8
    u.pi = NIL
    INSERT(Q, u)

DECREASE-KEY(Q, r, 0)

while Q != de l'ensemble vide
    u = EXTRACT-MIN(Q)

    for each v appartien a G.Adj[u]
        if v appartient a Q and w(u, v) < v.key
            v.pi = u
            DECREASE-KEY(Q, v, w(u, v))
```

Basic idea:

- Start from an arbitrary root vertex r

- Maintain a priority queue of vertices not yet in the tree
- Always pick the vertex with smallest connection cost (key)
- Update neighbors if a cheaper connection is found

Goal: build a Minimum Spanning Tree by greedy expansion

Time complexity:

- Initialization: $O(V)$
- Decrease-key on root: $O(\log V)$
- Extract-min: $O(V \log V)$
- Decrease-key operations: $O(E \log V)$

Total: $O(E \log V)$ (or $O(E + V \log V)$ with Fibonacci heap)

15.2 Kruskal's algorithm

Concept and idea:

Kruskal's algorithm finds a **Minimum Spanning Tree (MST)** by sorting all edges by increasing weight and adding them one by one, as long as they do not create a cycle.

It builds the MST globally (edge by edge), instead of growing it from a single source like Prim.

```
KRUSKAL(G, w)

A = ensemble vide

for each vertex v appartient a G.V
    MAKE-SET(v)

sort the edges of G.E into nondecreasing order by weight w

for each (u, v) taken from the sorted list
    if FIND-SET(u) != FIND-SET(v)
        A = A U {(u, v)}
        UNION(u, v)

return A
```

Basic idea:

- Start with each vertex in its own component
- Sort all edges by weight (cheapest first)
- Add an edge only if it connects two different components
- Avoid cycles using Union-Find

Goal: construct a Minimum Spanning Tree by globally selecting the smallest safe edges

Time complexity:

Initialize A: $O(1)$ First for loop: $O(V)$ Make-Sets Sorting edges: $O(E \log E)$ Second for loop: $O(E)$ Find-Set and Union operations

Total time: $O((V + E)\,\alpha(V)) + O(E \log E) = O(E \log E) = O(E \log V)$

If edges already sorted time is O(E * alpha(V)) which is almost linear

CHAPTER 16

THE SHORTEST PATH PROBLEM

16.1 NEGATIVE WEIGHTS AND APPLICATIONS

Negative Weights in Graphs

We will allow negative edge weights.
This is valid as long as there is **no negative-weight cycle reachable from the source**.
If a negative cycle exists, then we could loop around it indefinitely and keep decreasing the path cost without bound.

Why Negative Weights are a Problem

Some algorithms (e.g. Dijkstra's algorithm) only work correctly when all edge weights are non-negative.
This is because they assume that once a vertex is processed, its shortest path is final, which is false when negative edges exist.

Why Negative Weights are Useful

Negative weights are important because they model real-world situations where gains and losses exist.
En mots simples : les poids négatifs permettent de représenter des "profits" ou des réductions de coût, pas seulement des distances.

Example: Currency Exchange

Negative weights can represent currency exchange or arbitrage problems:

- vertices = currencies
- edges = exchange rates
- weights = cost or gain (log-transformed)

Idea: finding a negative cycle corresponds to finding a way to start with money and end up with more money after a sequence of exchanges.

16.2 Bellman ford algorithm

16.2.1 Init single source

Concept and idea:

The INIT-SINGLE-SOURCE procedure initializes the shortest-path algorithm by setting the initial distance values from a source vertex s.

All vertices are initially considered unreachable, except the source.

```
INIT-SINGLE-SOURCE(G, s)

for each v appartient a G.V
    v.d = 8
    v.pi = NIL

s.d = 0
```

Basic idea:

- Set all distances to infinity (unknown / unreachable at start)
- Remove all predecessors (no known path yet)
- Set source distance to 0

Goal: prepare the graph for shortest-path algorithms

Time complexity: $O(V)$

16.2.2 Relax

Concept and idea:

The RELAX operation is the core step in shortest-path algorithms. It tries to improve the best known distance to a vertex v by going through an adjacent vertex u.

If a shorter path is found, it updates the distance and remembers the predecessor.

```
RELAX(u, v, w)

if v.d > u.d + w(u, v)
    v.d = u.d + w(u, v)
    v.pi = u
```

Basic idea:

- Check if going through u gives a shorter path to v
- If yes, update the best known distance
- Store u as the predecessor of v

Goal: progressively improve shortest-path estimates

Time complexity: $O(1)$

16.2.3 Bellman-for main

Concept and idea:

The Bellman-Ford algorithm computes shortest paths from a single source in a weighted graph, even when negative weights are present (as long as there is no negative cycle reachable from the source).

It repeatedly relaxes all edges to progressively improve distance estimates.

```
Bellman-Ford(G, w, s)

Init-Single-Source(G, s)

for i = 1 to |G.V| - 1
    for each edge (u, v) appartient a G.E
        Relax(u, v, w)
```

Basic idea:

- Initialize all distances from the source
- Repeatedly relax all edges
- Each pass allows shortest paths with one more edge to be found
- After |V| - 1 iterations, all shortest paths are correct

Goal: compute shortest paths even with negative edge weights

Time complexity: $O(V \cdot E)$

16.2.4 Bellman-for main with negative cycle detect

Concept and idea:

The Bellman-Ford algorithm computes single-source shortest paths in a weighted graph and is also able to detect negative-weight cycles reachable from the source.

It first relaxes all edges repeatedly, then performs an extra pass to check for improvements.

```
Bellman-Ford(G, w, s)

Init-Single-Source(G, s)

for i = 1 to |G.V| - 1
```

```
    for each edge (u, v) appartient a G.E
        Relax(u, v, w)

for each edge (u, v) appartient a G.E
    if v.d > u.d + w(u, v)
        return FALSE

return TRUE
```

Basic idea:

- Initialize all distances from the source
- Relax all edges repeatedly to propagate shortest paths
- Do one final pass to check if any distance can still be improved
- If yes, a negative cycle exists

Goal: compute shortest paths and detect negative-weight cycles

Time complexity: $O(V \cdot E)$

16.3 Dijkstra algorithm

Concept and idea:

Dijkstra's algorithm computes single-source shortest paths in a weighted graph with **non-negative edge weights**.

It repeatedly selects the vertex with the smallest tentative distance and "finalizes" it.

```
DIJKSTRA(G, w, s)

INIT-SINGLE-SOURCE(G, s)

S = ensemble vide
Q = G.V

while Q != ensemble vide
    u = EXTRACT-MIN(Q)
    S = S U {u}

    for each vertex v appartient a G.Adj[u]
        RELAX(u, v, w)
```

Basic idea:

- Start from source with distance 0
- Maintain a priority queue of unprocessed vertices
- Always pick vertex with smallest current distance
- Relax all outgoing edges of the chosen vertex

- Once a vertex is extracted, its shortest path is finalized

Goal: compute shortest paths efficiently when all weights are non-negative

Time complexity:

- With binary heap: $O((V + E) \log V)$
- With adjacency list + heap: $O(E \log V)$

Running time:

Like Prim's algorithm, the running time is dominated by operations on the priority queue.

- If a binary heap is used: each operation costs $O(\log V)$ $\Rightarrow O(E \log V)$
- More careful implementation (with efficient decrease-key structure): $O(V \log V + E)$

16.4 Dijkstra algorithm

16.5 Ford-Fulkerson Method

Residual network

Given flow f, the residual network G_f has residual capacity $c_f(u, v) = c(u, v) - f(u, v)$ for forward edges and $c_f(v, u) = f(u, v)$ for back edges. An **augmenting path** is any s-t path in G_f.

```
FORD-FULKERSON(G, s, t):
    initialise f = 0 on all edges
    while exists augmenting path p in G_f:
        cf(p) = min capacity along p
        augment f along p by cf(p)
    return f
```

Max-flow min-cut theorem

The following are equivalent for a flow f:

1. f is a maximum flow.
2. There is no augmenting path in G_f.
3. $|f|$ equals the capacity of some minimum cut (S, T).

16.6 Applications

16.6.1 Bipartite matching via max flow

Algorithm

Goal: Maximum matching in a bipartite graph $G = (L \cup R, E)$.
Reduction:

- Add source s with edge $s \to u$ (capacity 1) for each $u \in L$.
- Keep all edges $u \to v$ for $(u, v) \in E$ (capacity 1).
- Add sink t with edge $v \to t$ (capacity 1) for each $v \in R$.

Run max-flow. Each unit of flow corresponds to one matched pair. The maximum matching has size $|f^*|$.

16.6.2 Edge-disjoint paths via max flow

Algorithm

Goal: Maximum number of s-t paths sharing no edge.
Reduction: Keep the original graph, set every edge capacity to 1. Run max-flow from s to t. The value $|f^*|$ equals the maximum number of edge-disjoint paths. By the max-flow min-cut theorem, it also equals the minimum number of edges whose removal disconnects s from t.

CHAPTER 17

Quizzes

17.1 Quiz #7

17.1.1 Bipartite matching via max flow

Bipartite matching via max flow — student notes

Cet algorithme est une application de max flow : même code en substance, mais le contexte et les règles du graphe changent.
Qu'est-ce que le bipartite matching ? On a deux sets d'éléments et le but est de faire le maximum de paires possibles, sachant que chaque élément ne peut être apparié qu'une seule fois avec un élément de l'autre set, et uniquement avec certains.
Modélisation du graphe :

src → tous les éléments du premier set (capacité 1)
éléments set 1 → éléments set 2 compatibles (capacité 1)
éléments set 2 → end (capacité 1)

La capacité 1 sur chaque branche garantit qu'elle n'est traversée qu'une fois, évitant ainsi les doublons ou les mauvais appariements.

17.1.2 Edge disjoint paths via max flow

Edge disjoint paths via max flow — student notes

On utilise max flow pour trouver le maximum de chemins de s à t sans intersection.
Qu'est-ce que edge disjoint paths ? On cherche le maximum de chemins de A à B qui ne partagent aucune arête.
Modélisation du graphe :

chaque chemin = arêtes du graphe
capacité de chaque arête = 1

La capacité 1 force chaque arête à n'être empruntée qu'une fois. Le max flow donne alors directement le nombre maximum de chemins arête-disjoints.

17.2 Quiz #9

17.2.1 Bellman-Ford

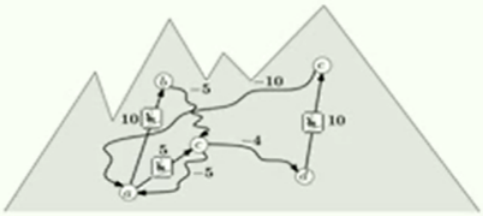
1. **Pourquoi $|V|-1$ itérations sont suffisantes :** Le nombre $|V|-1$ correspond au nombre maximal d'arêtes dans un chemin simple. Après $|V|-1$ itérations, l'algorithme a nécessairement trouvé tous les plus courts chemins ne contenant aucun cycle.
2. **Détection des cycles négatifs :** Si le coût d'un chemin peut encore être réduit lors de la $|V|$-ième itération, cela implique que le plus court chemin contient plus de $|V|-1$ arêtes. Ceci n'est possible que si le chemin contient un cycle, et pour que le coût diminue, ce cycle doit être négatif.

Problème d'examen vu en cours

Problem solving (exam)

Problem solving (20 pts * exam question ~~last~~ previous year)

The famous alpine ski racer Lindsey Vonn has sent her assistant to measure the altitude differences of the ski lifts and slopes of a famous ski resort in Switzerland. The assistant returns after several days of hard work with a map of all ski lifts and all slopes together with the altitude difference between the start station and the end station of each lift and the altitude difference between the start station and end station of each slope. A slope starts from the end station of a lift and ends at the start station of a potentially different lift (see the figure below).
Lindsey wants to verify the map of her assistant by performing the following sanity check: starting from any point A, no matter how we ski (using lifts and slopes), the altitude change when we return to A should be 0 (i.e., neither strictly negative nor strictly positive). Design an algorithm to perform this sanity check and analyze its running time in terms of the number of slopes and ski lifts ($|E|$) and the number of start and end stations ($|V|$). Your algorithm should run in polynomial time (in $|V|$ and $|E|$).



Idée. La carte est cohérente si et seulement si tout cycle C du graphe vérifie $W(C) = 0$. On décompose cette condition en deux :

$$\forall C : W(C) = 0 \iff (\forall C : W(C) \geq 0) \textbf{ ET } (\forall C : W(C) \leq 0)$$

Chacune se vérifie avec Bellman-Ford en détectant l'absence de cycle négatif : une fois avec les poids w, une fois avec $-w$.

Pourquoi $|V|-1$ itérations suffisent. Un plus court chemin sans cycle a au plus $|V|-1$ arêtes. Après $|V|-1$ itérations de relaxation, toutes les distances correctes sont calculées.

- **Pas de cycle négatif :** tout est stabilisé après $|V|-1$ itérations ; une $|V|$-ème ne relaxera plus rien.

- **Cycle négatif présent :** on peut indéfiniment réduire une distance en repassant par le cycle, donc la $|V|$-ème itération relaxe encore au moins une arête.

En exécutant Bellman-Ford deux fois (une fois avec w, une fois avec $-w$), la solution complète reste $O(|V| \cdot |E|)$.

17.2.2 Dijkstra

Ce qu'il faut retenir

L'idée centrale. A chaque étape on finalise le sommet non-visité dont la distance estimée est minimale. Cette distance est déjà optimale — car tous les poids sont ≥ 0, aucun chemin futur ne pourra la réduire. C'est exactement ce qui distingue Dijkstra de Bellman-Ford : pas de deuxième chance, pas de retour en arrière.

Ce qui se passe à chaque itération.

1. On extrait $u = \arg\min_{v \in Q} v.d$ de la file.
2. On l'ajoute à S — sa distance est maintenant définitive.
3. Pour chaque voisin v de u : si $u.d + w(u, v) < v.d$, on met à jour $v.d$ *et* $v.\pi$.

Pourquoi ça ne marche pas avec des poids négatifs. Si une arête négative (x, y) existe, un sommet x déjà dans S pourrait offrir un chemin plus court vers y mais y est peut-être déjà finalisé.